

Sistemas Operativos

2º Parcial 2C2025 – TT – Resolución

Aclaración: La mayoría de las preguntas o ejercicios no tienen una única solución. Por lo tanto, si una solución particular no es similar a la expuesta aquí, no significa necesariamente que la misma sea incorrecta. Ante cualquier duda, consultar con el/la docente del curso.

Teoría – Single choice: elija para cada pregunta la opción correcta (solo una es válida)

- 1- Reubicación
- 2-Segmentación
- 3-Reduce el espacio ocupado en la memoria
- 4- Offset del segmento
- 5- Modo de apertura
- 6- La dirección del primer cluster
- 7- El número de inodo

Teoría – Desarrollar: responda las siguientes preguntas justificando sus respuestas

1- Particiones fijas posee fragmentación interna (aún más grave que paginación ya que el tamaño de la partición suele ser más grande porque hay que alojar al proceso de manera contigua), no es necesario compactar ya que no posee fragmentación externa y limita el grado de multiprogramación en el sistema ya que no puede haber más procesos que las N particiones definidas y además limita el tamaño máximo de un proceso ya que no puede ser más grande que el tamaño de partición.

Por otro lado, particiones dinámicas no posee fragmentación interna pero sí “requiere” de compactación para solucionar la fragmentación externa. No limita ni el grado de multiprogramación ni el tamaño de un proceso, ya que las particiones son asignadas de forma dinámica.

2 – Para que ocurra un page fault, la MMU obtiene a qué página corresponde la dirección lógica generada por el proceso y busca la misma en la TLB (de existir) y en las tabla de páginas de dicho proceso. Cuando la Presencia de la página se encuentra en 0, la MMU misma es la encargada de generar la interrupción de Page Fault. La misma será atendida por el Kernel, que se encargará eventualmente de cargar dicha página en memoria y actualizar las estructuras necesarias.

3 –

- a) Verdadero. Un softlink es una nueva entrada de directorio que apunta a un nuevo archivo (con su propio inodo), que contiene la ruta del archivo que es apuntado por este.
- b) Falso. No es posible realizar un hardlink a un archivo en otro volumen, independientemente del File System utilizado por este.

Práctica - Desarrollar: resuelva los siguientes ejercicios justificando sus respuestas

1 - Worst Fit Asigna el nuevo segmento al espacio libre más grande disponible:

Espacio libre 1: Base: 4096, tamaño 3072

Espacio libre 2: Base: 9216, tamaño 1024

Espacio libre 3: Base: 12288, tamaño 2048

Los elegidos, en orden, serán el **EL1** y el **EL3**.

Tabla de Segmentos del Proceso 2:

	Base	Tamaño
Segmento 1	4096	2048
Segmento 2	12288	1024

2 -

F	PID P U M	PID P U M	PID P U M	PID P U M	PID P U M	
	Pag 1 - PC (L)	Pag 0 - PA (E)	Pag 5 - PC (E)	Pag 4 - PB (E)	Pag 2 - PA (L)	
0	PB - 1 00	PB - 1 00	PC - 5 11	PC - 5 11	PC - 5 11	
1	PB - 5 11	PB - 5 11	PB - 5 11	PB - 5 01	PB - 5 01	
2	PA - 2 10	PA - 2 10	PA - 2 10	PA - 2 00	PA - 2 10	
3	PC - 1 10	PC - 1 10	PC - 1 10	PC - 1 00	PC - 1 00	
4	PB - 3 01	PA - 0 11	PA - 0 11	PA - 0 01	PA - 0 01	
5	PC - 4 10	PC - 4 10	PC - 4 00	PB - 4 11	PB - 4 11	
6	PA - 4 11	PA - 4 11	PA - 4 01	PA - 4 01	PA - 4 01	
7	PC - 2 11	PC - 2 11	PC - 2 01	PC - 2 01	PC - 2 01	
PF		1	1	1	0	3
AD		2	1	1	0	4

Aclaración: La celda amarilla indica donde se hizo el reemplazo (o último reemplazo si se referenció a una página ya cargada), el puntero queda en la siguiente posición.

El algoritmo empleado es CLOCK debido a que los reemplazos los hace fijándose en qué frame encuentra una página cargada con su bit de uso en cero, de esta forma la selecciona como víctima. Además, a medida que recorre la lista de frames, si su bit es 1, lo cambia a 0.

Si bien en los 2 primeros reemplazos podría haber sido LRU, no lo es debido a que en el 3er reemplazo tuvo que reemplazar la página 2 del Proceso C, la cual se encontraba en el frame 7.

FIFO tampoco puede ser porque la forma de reemplazo no es frame a frame. Podemos ver que para el primer reemplazo si lo es, pero luego no se reemplaza a la página del frame 5.

CLOCK MODIFICADO tampoco es porque primero se debe buscar un 00 (bit de uso = 0 y bit de modificado = 0), y lo hubiese encontrado en el frame 0 (tenía cargada la página 1 del proceso B). El reemplazo no fue realizado en dicho frame, siendo esta la razón por la cual no es una posible solución a la interrogante planteada.

3-

- a) Tam. Real del FS: cant. punt. * tam. bloque: $2^{28} * 2^{11} \text{ B} = 2^{39} \text{ B} = 512 \text{ GiB}$.
Tam. Real del Archivo: cant. bloques * tam. bloque - Frag. int = $4 \text{ bloques} * 2^{11} \text{ B} - 512 \text{ B} = 7,5 \text{ KiB}$
- b)

...	...
4	6
...	...
6	8
...	...
8	10
...	...
10	FIN
...	...

c)

4-

Nro de bytes inicial: 6.553.600
Nro de bytes final: 8.631.975.936

Si cada uno de ellos lo divido por el tamaño de página ($8 \text{ KiB} = 2^{13} \text{ bytes}$), obtengo los bloques donde se encuentran cada uno de ellos:

Bloque de datos inicial para la lectura: $6.553.600 / 2^{13} = \text{bloque de datos nro } 800$

Bloque de datos final para la lectura: $8.631.975.936 / 2^{13} = \text{bloque de datos nro } 1.053.708$

Luego, debemos calcular cuántos son los bloques a leer del archivo:
 $1.053.708 - 800 + 1 = 1.052.909 \text{ bloques}$ (se le suma 1 ya que el rango incluye el bloque 800 y el 1.053.708)

También menciona que los punteros son de 64 bits, por lo tanto, debo transformarlo a bytes $64 / 8 = 8 \text{ bytes}$
Con este dato, sabemos la cantidad de punteros que tendremos por cada bloque de punteros:
 $2^{13} / 2^3 = 2^{10} = 1024 \text{ punteros}$

Como debemos leer a partir del bloque 800 de datos, los primeros 12 punteros directos no los vamos a acceder. Es por eso que vamos a acceder al puntero indirecto simple, del cual accederemos solamente a los últimos 236 punteros directos. Esto sale de realizar $12 + 1024 - 800 = 236$

De momento solo accedimos a 236 punteros directos (pertenecientes al bloque de punteros referenciado por el puntero indirecto simple), significando que referenciamos a 236 bloques de datos.

Calculamos lo restante: $1.052.909 - 236 = 1.052.673 \text{ bloques nos restan por leer}$

Ahora sabemos que vamos a tener que acceder a un puntero indirecto doble como mínimo. Dentro del bloque de punteros al que referencia, este tiene 1024 punteros indirectos simples. Luego, va a poder referenciar hasta $1024^2 \text{ bloques de datos}$ como máximo.

Si verificamos la cuenta: $1.052.673 - 1024^2 = 4.097 \text{ bloques restantes por leer}$

Es decir, tuvimos que acceder a todos los bloques de datos referenciados por este primer puntero indirecto doble.

Posteriormente, sabemos que vamos a tener que acceder al otro puntero indirecto doble del que disponemos. Lo que nos interesa es calcular cuántos punteros indirectos simples vamos a tener que referenciar para poder cubrir la cantidad de bloques de datos restantes por leer.

Para ello, podemos realizar la siguiente cuenta: $4.097 / 1.024 = 4,001$. Eso quiere decir que tengo que acceder a 5 indirectos simples dentro de ese doble.

Finalmente, sabemos la cantidad de bloques de datos y bloques de punteros que tuvimos que acceder para leer la información del archivo solicitada, siendo la misma de 1.052.909 bloques de datos y 1.032 bloques de punteros (Esto último sale de hacer 1 ind. simple + 1 ind. doble + 1024 ind. simples + 1 ind. doble + 5 ind. simples).

5-

a)

Byte modificado: 4.300.801

Tamaño de página: 4096

nro página = $4300801 / 4096 = 1050, \dots = 1050$

offset en página = 1

Con páginas de 4KiB, el tamaño del offset es 12 bits.

Con 16 segmentos por proceso, el tamaño del número de segmento es 4 bits.

Total 32 bits - 12 bits - 4 bits = 16 bits para número de página

Segmento usado por el bloc de notas: 4

Dirección lógica final:

(seg = 4, página = 1050, offset = 1) → 4041A001h (0A1Ah es 1050 decimal en hexa)

Para obtener dicha dirección lógica se accede a la TLB, tabla de segmentos y por último a la tabla de páginas.

b)

En EXT2 tenemos bloques de 4KiB y punteros de 4 bytes. Tenemos 1024 punteros x bloque de punteros.

Con los punteros directos accedemos a los primeros 12 bloques, con el puntero indirecto simple accedemos a los próximos 1024 pero para acceder el número 1050 hay que utilizar el puntero indirecto doble. Es decir que en definitiva necesitamos 3 accesos, 1 para acceder al bloque indirecto doble, 1 para el simple y 1 para el bloque de datos.

c) En el inodo se ven reflejados cambios sobre la estructura del archivo. Al no verse modificado su tamaño, permisos, propietario o referencias, el único cambio que se podría mencionar es el timestamp de último acceso y última modificación.